

Bayesian optimisation

Find the maximum of an expensive function in few evaluations: fit a surrogate with error bars, then let a cheap acquisition function pick each next point.

THE QUESTION

When each evaluation of f costs a training run and returns no gradient, where should the next one go?

THE INTUITION

Fit a probabilistic surrogate to the points seen so far. Its mean says where f looks high, its spread where we know little. An acquisition function scores every x on both, and we evaluate its maximiser.

KEY TAKEAWAY

$x_{t+1} = \arg \max_x \alpha(x)$ under the posterior.
 $\text{PI } \Phi(z) \cdot \text{EI } (\mu - y^+ - \xi)\Phi(z) + s\varphi(z) \cdot \text{UCB } \mu + \kappa s$.
 Some exploration and a sensible prior keep the search from stalling.

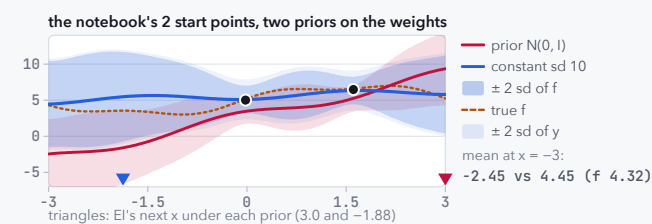
data D_t → surrogate $p(f | D_t)$ → acquisition $\alpha(x)$ → $x_{\text{next}} = \arg \max \alpha$ → pay for $y = f(x) + \epsilon$ → add to D , repeat

1 Expensive, gradient-free, maybe noisy: the black-box setting



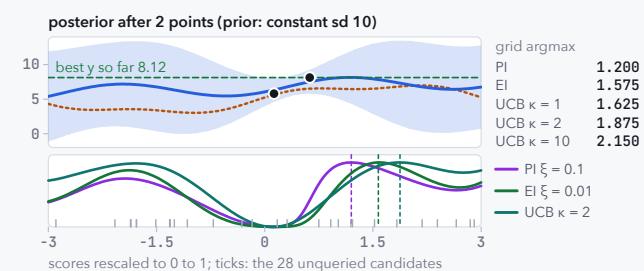
Goal: $x^* = \arg \max_{x \in A} f(x)$ on a simple domain A (a box). The older deck's conditions: f has no structure to exploit (not concave), gives no gradient, is expensive to evaluate, and may be noisy, $y = f(x) + \epsilon$ with Gaussian ϵ . Left (quoted): tuning one L2 rate of an MLP took 11 full trainings. Right: the notebook's test function; its candidates are 30 points $x \sim U(-3, 3)$ with noise sd 1.

2 The surrogate is Bayesian linear regression, and its prior mean matters



The notebook's model: features $\varphi(x) = [1, x, \sin 2x, \cos 2x, \sin(2x + 0.5), \cos(2x + 0.5)]$, noise $\sigma = 1$,
 $\mu(x) = \varphi^T m_N$, $s^2(x) = \varphi^T S_N \varphi$, $\text{Var}[y] = \sigma^2 + s^2$
 Acquisitions score f , so they use $s(x)$. This is a GP with kernel $\varphi(x)^T S_0 \varphi(x')$; the older deck uses a Matérn GP. The two points are the notebook's own, recovered from its printed m_N and S_N . Its prior $\mathcal{N}(0, I)$ says $f(x) \approx 0 \pm \sqrt{3 + x^2}$, but f lives near 5. Away from the data the mean sinks (-2.45 at $x = -3$), the x feature extrapolates to 9.36 at $x = 3$, and EI goes to that edge. A prior sd of 10 on the constant weight (or centring y) fixes the level: EI moves to -1.88 , and over 200 pools EI's gap after 10 rounds falls from 0.183 to 0.047 (panel 6). The rest of the sheet uses it. The notebook's plot draws mean $\pm 2(1/\sigma^2 + s^2)$, a variance as a half-width: 14.9 at $x = 3$ instead of 5.46.

3 PI, EI and UCB read one posterior differently



With y^+ the best observed y so far and $z = (\mu(x) - y^+ - \xi)/s(x)$:

$$\alpha_{\text{PI}} = \Phi(z), \quad \alpha_{\text{UCB}} = \mu + \kappa s,$$

$$\alpha_{\text{EI}} = (\mu - y^+ - \xi)\Phi(z) + s\varphi(z)$$

PI is the right-tail mass above $y^+ + \xi$; it peaks next to the best mean (1.2; the mean peaks at 1.175). EI goes a little further (1.575), and UCB moves toward the wide band as κ grows. All three are cheap to maximise: a grid here, restarts of a local optimiser in more dimensions. Update y^+ every round. The notebook calls κ "beta" and ξ "eps".

4 EI is the expected gain over y^+ , in closed form



With $f(x) \sim \mathcal{N}(\mu, s^2)$, threshold $\tau = y^+ + \xi$ and $z = (\mu - \tau)/s$:

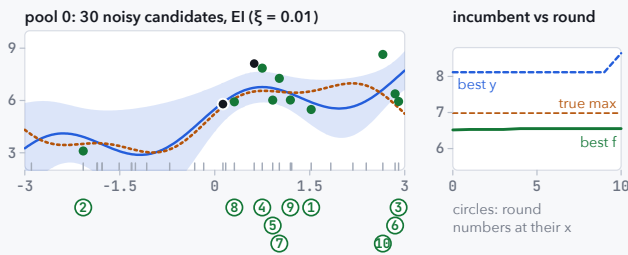
$$\begin{aligned} \mathbb{E}[\max(0, f - \tau)] &= \int_{\tau}^{\infty} (f - \tau) \mathcal{N}(f | \mu, s^2) df \\ &= (\mu - \tau)\Phi(z) + s\varphi(z) \end{aligned}$$

Substitute $f = \mu + su$; the u part integrates to $s\varphi(z)$. The first term rewards a high mean (exploit), the second a wide band (explore). With $\xi = 0$: at $x = 1.2$ PI is higher (0.51 against 0.46); at $x = 1.575$ the wider band wins on EI (0.93 against 0.78). Monte Carlo agrees within 0.002. $\Phi(z) = (1 + \text{erf}(z/\sqrt{2}))/2$ needs only `math.erf`.

Explore enough, and sample whole functions

A worked run, a comparison with random search, the exploration knob, and Thompson sampling with one posterior draw per round.

5 A worked run: EI settles on the plateau, not the peak



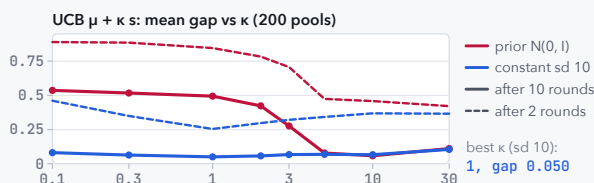
Pool 0 of the seeded pools, not picked for effect. Rounds 2 and 3 explore $x = -2.08$ and 2.90 ; from round 4 the best f is 6.554 at $x = 0.75$, and the pool's best (6.983 at 2.18) is never queried. The incumbent $y^+ = 8.116$ is a noisy y where $f = 6.52$, above the true maximum 6.984; so is the notebook's reported best, 7.997 (quoted).

6 Over 200 pools, BO beats random search once the prior fits



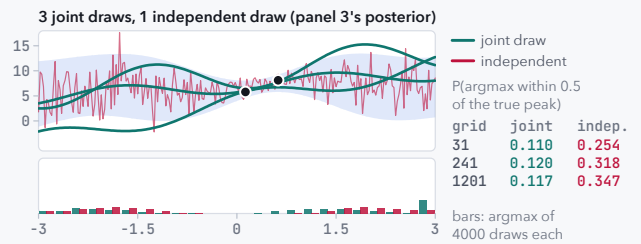
Pools of 30 candidates (noise sd 1), 2 start points, 10 rounds; gap = pool's best f minus best f queried, stuck = gap above 0.5; random search averaged over 20 seeds per pool. With the sd-10 prior, PI and EI end at 0.038 and 0.047 against random's 0.097; with the notebook's $\mathcal{N}(0, I)$, EI trails random throughout. Random sees 40% of each pool.

7 κ trades early gains against getting stuck



With $\mathcal{N}(0, I)$, UCB at $\kappa = 0.1$ is stuck in 19.5% of pools; $\kappa = 10$ cuts the gap to 0.058 by exploring past the wrong prior. With sd 10 the curve is U-shaped: $\kappa = 30$ wastes rounds on the edges. EI's ξ acts more gently. Quoted: with $\beta = 10$ the notebook's UCB tries $x = -0.907$ and -2.469 ; with $\beta = 0.1$ it stays above 0.3.

8 Thompson sampling draws one whole function per round



Draw one weight vector $\theta \sim \mathcal{N}(m_N, S_N)$, form $g(x) = \varphi(x)^T \theta$ and query its argmax: each x is queried with the posterior probability that it is the maximiser. The notebook (cell 44) draws each x on its own from $\mathcal{N}(\mu, \sigma^2 + s^2)$, seed reset each call: neighbours are uncorrelated (-0.007 against 0.999), and the argmax depends on the grid (table). On these pools it does no worse (gap 0.046 against 0.069), but it samples noise, not f .

9 Bayesian optimisation in NumPy

```
from math import erf, sqrt, pi
from numpy.linalg import inv
Phi = np.vectorize(lambda z:
    0.5 * (1 + erf(z / sqrt(2))))
def ei(mu, s, yb, xi=0.01):
    d = mu - yb - xi; z = d / s
    pdf = np.exp(-z**2 / 2) / sqrt(2*pi)
    return d * Phi(z) + s * pdf
S0 = np.eye(6); S0[0, 0] = 100
idx, n = [0, 1], len(X) # 2 start points
for t in range(10): # noise sd 1
    P = phi(X[idx])
    SN = inv(inv(S0) + P.T @ P)
    mN = SN @ P.T @ Y[idx]
    rest = np.setdiff1d(np.arange(n), idx)
    Q = phi(X[rest])
    s = np.sqrt((Q @ SN * Q).sum(1))
    a = ei(Q @ mN, s, Y[idx].max())
    # Thompson: a = Q @ th, th ~ N(mN, SN)
    idx.append(rest[np.argmax(a)])
```

10 Pause and think

Can these six features represent f ? No. Two are combinations of $\sin 2x$ and $\cos 2x$ (rank 4), and f 's $\sin x$ and $\sin 3x$ lie outside the span: the best fit in it misses f by up to 2.17. How does this differ from active learning? Active learning queries the largest $s(x)$ to learn f everywhere (UCB as $\kappa \rightarrow \infty$); BO spends evaluations where the maximum might be.

Surrogate with error bars $\rightarrow$ maximise a cheap acquisition $\rightarrow$ evaluate f there · a wrong prior or too little exploration stalls it

Source: lecture slides "Bayesian optimization" (one frame), the older deck on Bayesian optimisation with a GP, and the notebooks on Bayesian optimisation and its motivation, nipunbatra.github.io/pml-teaching. Quoted: panel 1 (left), notebook runs in panels 5 and 7. Else NumPy, fixed seeds. Corrections: the feature docstring says $[1, x, \sin x, \dots]$, the code builds $\sin 2x, \cos 2x$ and shifts; the band plot uses $\pm 2(1/\sigma^2 + s^2)$ for $\pm 2\sqrt{\sigma^2 + s^2}$; Thompson draws independent marginals; the loop never updates y^+ . Older deck: PI is the right tail, not the left; $\alpha_{PI} = P(f(x) \geq f(x^+) + \varepsilon)$ (misplaced bracket); "argmax $f(x)x$ "; EI's Z undefined.